Chandigarh University

STUDENT MANAGEMENT SYSTEM

A DBMS-Based PBL, Case Study and Mini Project

Project Title: Student Management System Using MySQL

Submitted by	Kavya
UID	25BCD10086
Programme	BCA Data Science
University	Chandigarh University
Academic Session	2026–2027

Table of Contents

No.	Section
–	Certificate
–	Acknowledgement
–	Abstract
1	Introduction
2	Theoretical Background
3	Problem Statement and Need for the Study
4	Objectives and Scope
5	Case Study — Student Record Management
6	Proposed System and Its Benefits
7	Feasibility Study
8	Modules and Functional Requirements
9	Non-Functional Requirements
10	DBMS Concepts Used in the Project
11	ER Model and Relationships
12	Relational Schema and Data Dictionary
13	Normalization and Data Integrity
14	Tools, Technologies and Methodology
15	Mini Project — Database and Table Creation
16	Mini Project — Retrieval and Updating
17	Mini Project — Filtering, Aggregation and Joins
18	Mini Project — Delete Operation and Final Results
19	Testing and Output Verification
20	Results, Limitations and Future Scope
21	Conclusion and References

CERTIFICATE

This is to certify that the project report entitled “Student Management System Using MySQL” has been prepared as part of the academic requirements of the Database Management System course.

The project presents a case study of student record management and demonstrates the practical application of relational database concepts using MySQL. It includes theoretical analysis, database design, SQL implementation, testing, and documentation of the expected results.

The project has been prepared by the student under the guidance of the concerned faculty member.

Student Name	Kavya
UID	25BCD10086
Programme	BCA Data Science
Faculty Signature	_____
Date	_____
Department/University	_____

Note: Obtain the faculty's approval and signature before including this certificate in the final submission.

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my faculty members for providing me with the opportunity to undertake this Database Management System project titled “Student Management System Using MySQL.”

This project has helped me understand the practical importance of database management in educational institutions. Through this work, I gained knowledge of relational database design, SQL commands, data integrity, constraints, normalization, and the retrieval of information from multiple tables.

I am also thankful to Chandigarh University for providing the academic environment and learning resources necessary to complete this project. I would like to acknowledge the developers of MySQL, Visual Studio Code, and GitHub for providing tools that support database development, testing, and project documentation.

Finally, I am grateful to everyone who provided guidance and encouragement during the preparation of this project report.

ABSTRACT

The Student Management System is a database-oriented project designed to demonstrate how student information can be stored, organized, updated, and retrieved using a relational database management system. Educational institutions manage large amounts of information, including student details, course registrations, academic records, and performance data. Handling these records manually may result in duplication, inconsistency, and difficulties in retrieving accurate information.

The proposed project uses MySQL to develop a structured database for managing student and course information. It consists of two related tables: Students and Courses. The Students table stores student identification numbers, names, email addresses, ages, course references, and marks. The Courses table stores course identification numbers and course names.

The project implements fundamental SQL operations, including database creation, table creation, data insertion, retrieval, updating, deletion, sorting, filtering, aggregate functions, and joins. It also demonstrates primary keys, foreign keys, unique constraints, check constraints, default values, and relational integrity.

The project follows a practical, step-by-step methodology, beginning with problem identification and database design and progressing to implementation and testing. Sample records are used to demonstrate how the system behaves under different SQL operations.

The main purpose of this project is educational. It provides practical experience in designing and implementing a relational database while illustrating how database technology can improve the organization and management of student information.

Keywords: Database Management System, MySQL, SQL, Student Records, Relational Database, Primary Key, Foreign Key, Normalization, Data Integrity.

1. INTRODUCTION

1.1 Introduction to the project

A Student Management System is an information management system used to maintain student-related data in a structured and organized manner. It provides a systematic approach to storing student details and retrieving information whenever required.

Educational institutions generate and maintain a significant amount of information throughout an academic year. This information may include student names, identification numbers, contact details, course enrolments, examination marks, attendance records, and other academic details. Managing these records efficiently is essential for the smooth functioning of an institution.

A manual record management system generally depends on registers, paper documents, or separate spreadsheets. As the number of students increases, maintaining these records becomes more difficult. Searching for individual records may take time, and updating information across multiple documents can introduce inconsistencies.

A database management system provides a better approach by storing related information in structured tables. SQL can then be used to add, modify, delete, and retrieve the required data.

The Student Management System developed in this project uses MySQL, a relational database management system. It maintains student information in one table and course information in another. The tables are connected through a common course identification number, demonstrating the relational nature of a database.

1.2 Background of the study

The development of computerized record management systems has changed how organizations store and process information. In educational institutions, digital databases can support administrative tasks by maintaining consistent records and making information easier to retrieve.

A relational database organizes information into tables consisting of rows and columns. Each row represents an individual record, while each column represents an attribute of that record. Relationships between tables are established through keys.

In this project, each student is assigned a unique student ID, while each course is assigned a unique course ID. The course ID stored in the Students table references the corresponding record in the Courses table.

This design separates student information from course information. Consequently, a course name does not need to be repeatedly stored as a separate text value for every student. The database can retrieve the course name through a JOIN operation.

The project uses a small sample dataset so that the database structure and SQL operations remain easy to understand. However, the underlying concepts are also relevant to larger database applications.

1.3 Importance of database management in education

Database management is important in education for several reasons.

First, it helps maintain structured and consistent student records. Second, it allows administrators to retrieve information using conditions such as student ID, course, or marks. Third, it supports data integrity through constraints that prevent certain invalid entries.

Database systems also make it possible to generate summaries. For example, an institution may want to count students in each course or calculate average marks. SQL aggregate functions can perform these calculations without manually processing each record.

Therefore, database technology can improve the efficiency and reliability of information management in educational environments.

2. THEORETICAL BACKGROUND

2.1 Meaning of a Database

A database is an organized collection of related data that can be stored, accessed, and managed efficiently. It provides a structured way of maintaining information so that it can be retrieved and modified when required.

For example, a student database may contain student IDs, names, email addresses, course IDs, and examination marks. Instead of maintaining this information in separate paper records, it can be organized into database tables.

2.2 Meaning of DBMS

A Database Management System (DBMS) is software that allows users to create, organize, retrieve, update, and manage databases.

A DBMS acts as an interface between the user or application and the stored data. It provides mechanisms for defining database structures, executing queries, enforcing constraints, and controlling access.

Examples of DBMS software include MySQL, PostgreSQL, Oracle Database, Microsoft SQL Server, and SQLite.

In this project, MySQL is used because it supports relational tables, standard SQL operations, keys, constraints, joins, and aggregate functions.

2.3 Relational Database Management System

A Relational Database Management System (RDBMS) stores data in tables and establishes relationships between those tables.

Each table contains rows and columns. A row represents one record, and a column represents a particular property of that record.

The proposed project contains two tables:

1. **Students:** Contains student details and marks.
2. **Courses:** Contains course IDs and course names.

The course ID connects the two tables. This relationship allows the database to retrieve the name of the course in which a student is registered.

2.4 Characteristics of a DBMS

The major characteristics of a DBMS are:

- **Structured data storage:** Information is stored in an organized format.
- **Reduced duplication:** Proper database design reduces unnecessary repetition of information.
- **Data integrity:** Constraints help prevent invalid or inconsistent records.
- **Data retrieval:** SQL queries allow users to search for specific information.
- **Data security:** Database permissions can restrict who is allowed to access or modify data.
- **Data sharing:** Multiple authorized users and applications can work with the same database.
- **Backup and recovery:** Database administration tools can help restore data after failures when suitable backup procedures are configured.
- **Data independence:** Changes in the internal storage of data can often be made without requiring major changes to the applications using it.

2.5 Role of SQL

Structured Query Language (SQL) is used to communicate with relational databases. It provides commands for creating database objects, manipulating records, retrieving information, and controlling transactions and permissions.

The main SQL command categories are:

Category	Full Form	Purpose / Examples
DDL	Data Definition Language	Defines database structures. Examples include CREATE, ALTER, and DROP.
DML	Data Manipulation Language	Adds or modifies records. Examples include INSERT, UPDATE, and DELETE.
DQL	Data Query Language	Retrieves records using SELECT.
DCL	Data Control Language	Manages permissions using commands such as GRANT and REVOKE.
TCL	Transaction Control Language	Manages transactions using commands such as COMMIT and ROLLBACK.

The practical section of this project primarily demonstrates DDL, DML, and SELECT queries. DCL and TCL are explained as part of the theoretical study but are not required for the main sample workflow.

3. PROBLEM STATEMENT AND NEED FOR THE STUDY

3.1 Problem statement

Student records are essential to the administrative and academic activities of educational institutions. These records need to be stored accurately and retrieved efficiently. However, a manual or poorly organized record management approach may create problems when information needs to be searched, updated, or analyzed.

For example, an administrator may need to locate a particular student's record, identify all students enrolled in a course, or calculate the average marks of a group. If the information is maintained across separate registers or inconsistent files, completing these tasks may require considerable time.

The problem addressed by this project is the need for a simple and structured database that can store student information, maintain relationships with course records, and support common record management operations.

The proposed solution is a MySQL-based Student Management System that uses relational tables and SQL queries to manage sample student records.

3.2 Existing system

In a traditional system, student information may be recorded in paper registers or spreadsheets. Separate files may be used for student details and course information.

Such an approach may be sufficient for a small number of records. However, as the volume of information increases, it becomes more difficult to maintain consistency and retrieve information quickly.

Common limitations of the existing approach include:

1. Repeated entry of the same information.
2. Difficulty in searching for specific records.
3. Possibility of incorrect or outdated details.
4. Lack of consistent validation.
5. Time-consuming preparation of summary reports.
6. Difficulty in maintaining relationships between student and course records.
7. Limited ability to enforce data integrity across separate files.

3.3 Need for the proposed system

A structured database can address many of these limitations by organizing information into related tables.

The proposed system provides a single database containing student and course records. It uses unique identifiers to distinguish records and constraints to enforce basic data rules.

The system also supports SQL-based searching, sorting, updating, deletion, and summary calculations. It therefore provides a practical demonstration of how database technology can support routine administrative tasks.

3.4 Research questions for the case study

This case study is guided by the following questions:

1. How can student records be organized efficiently using a relational database?
2. How can primary keys and foreign keys maintain relationships between tables?
3. How can SQL constraints reduce invalid data entry?
4. How can queries retrieve student information and course details together?
5. How can aggregate functions support basic academic analysis?
6. What limitations remain in a simple database-only implementation?

These questions guide the theoretical discussion and the practical SQL implementation.

4. OBJECTIVES AND SCOPE

4.1 Primary objective

The primary objective of the project is to design and implement a basic Student Management System using MySQL and demonstrate the practical application of relational database concepts.

4.2 Specific objectives

The specific objectives are:

1. To create a database for maintaining student information.
2. To create related tables for students and courses.
3. To assign primary keys for unique record identification.
4. To establish a foreign key relationship between the tables.
5. To apply NOT NULL, UNIQUE, CHECK, and DEFAULT constraints.
6. To insert sample records into the database.
7. To retrieve all records and search for individual students.
8. To update student marks and verify the changes.
9. To filter and sort students according to their marks.
10. To calculate average marks using aggregate functions.
11. To combine student and course information using JOIN.
12. To count students enrolled in each course.
13. To demonstrate deletion of a student record.
14. To document query results using screenshots.
15. To upload the project files to GitHub for version control and academic submission.

4.3 Scope of the project

The project focuses on the basic database operations required to maintain student and course records. It is designed as an academic prototype rather than a complete institution-wide information system.

The current scope includes:

- Student identification and basic personal information.
- Course identification and course names.
- A student's association with a course.
- A numerical marks field.
- Basic validation through constraints.
- Record insertion, retrieval, updating, and deletion.
- Query-based academic summaries.
- Documentation of implementation and testing.

The current implementation does not include user authentication, a graphical interface, attendance management, fee collection, or a complete examination management module.

4.4 Intended users

The conceptual users of a Student Management System may include:

- **Administrative staff:** To maintain student records.
- **Faculty members:** To access permitted academic information.
- **Database administrators:** To manage database structure, security, and backups.
- **Students:** In an extended system, students could view their own permitted information through a secure interface.

The present prototype is executed directly through SQL and does not implement separate user roles.

4.5 Expected outcomes

At the end of the project, a functioning MySQL database should contain student and course tables, valid relationships, and sample records.

The SQL queries should demonstrate successful retrieval, modification, filtering, aggregation, and deletion. The results should be verified through actual query execution and captured as screenshots for the project report.

5. CASE STUDY — STUDENT RECORD MANAGEMENT

5.1 Overview of the case study

This case study considers a hypothetical educational department that needs to maintain records for students enrolled in different courses.

For the purpose of demonstration, the department has five students associated with three courses: BCA Data Science, BCA, and BBA.

The sample system stores each student's name, email address, age, course reference, and marks. Course names are maintained separately in the Courses table.

The dataset is illustrative and does not represent actual institutional records.

5.2 Scenario

Suppose an administrator receives the following requests:

- Register a new student.
- Display all student records.
- Find a student using the student ID.
- Correct the marks recorded for a student.
- Identify students scoring above 80 marks.
- Calculate the average marks of all students.
- Display student names with their course names.
- Count the students registered in each course.
- Remove a record when deletion is authorized.

The database must support these tasks without requiring the administrator to manually search multiple documents.

5.3 Data requirements

The system requires a unique ID for every student and a unique ID for every course.

A student record must have a name. An email address, if provided, must not duplicate another non-NULL email value. The age must be at least 16 according to the sample project's validation rule. Marks are represented using a decimal data type.

The course reference in the Students table must correspond to a course that exists in the Courses table. This ensures that a student cannot be assigned to an unknown course while the foreign key constraint is active.

5.4 Assumptions

The project uses the following assumptions:

1. Each student has one student ID.
2. Each course has one course ID.
3. Each student is assigned to one course in this simplified design.
4. Each student has one marks value for demonstration.
5. The sample age restriction is an academic example, not a universal institutional policy.
6. The marks field represents a numerical score, with no separate subject or examination table.
7. Only authorized users should modify or delete real institutional records.

These assumptions keep the database simple and make the project suitable for a beginner-level DBMS practical.

5.5 Analysis of the case study

The scenario shows that many routine administrative tasks can be performed using a small relational database.

A SELECT query can retrieve student details, while a WHERE condition can locate a specific record. UPDATE can correct a score, and aggregate functions can calculate average marks. JOIN operations can combine student and course information, and GROUP BY can generate course-wise counts.

The case study therefore provides a practical context for learning how relational database design and SQL solve structured information management problems.

6. PROPOSED SYSTEM AND ITS BENEFITS

6.1 Proposed system

The proposed system is a relational database named StudentManagement. It contains two tables: Students and Courses.

The Courses table is created first because the Students table contains a foreign key referencing it. Once the course records exist, student records can be inserted with valid course IDs.

The database supports the following operations:

1. Creating the database and tables.
2. Inserting course and student records.
3. Displaying stored records.
4. Searching by student ID.
5. Updating marks.
6. Filtering students according to their scores.
7. Calculating the average marks.
8. Combining student and course details.
9. Counting students in each course.
10. Deleting a student record.

6.2 Benefits

- **Improved organization:** Student and course data are stored in structured tables.
- **Efficient retrieval:** SQL queries can locate records without manually searching through documents.
- **Data consistency:** Keys and constraints help maintain valid relationships and basic data rules.
- **Reduced duplication:** Course information is stored separately from student information.
- **Flexible reporting:** Sorting, filtering, joins, and aggregate functions support different information requirements.
- **Easy modification:** SQL UPDATE can change selected records without recreating the complete table.
- **Practical learning:** The system provides hands-on experience with core database concepts.
- **Extensibility:** Additional tables can be introduced when more complex requirements arise.

6.3 Comparison with a manual system

Feature	Manual record system	Proposed database
Record storage	Registers or separate files	Relational tables
Searching	Manual lookup	SQL queries
Updating	Manual corrections	UPDATE statements
Validation	Depends on the person entering data	Database constraints
Summary reports	Manual calculation	Aggregate functions
Related information	May require separate documents	JOIN operations
Record deletion	Physical or manual removal	DELETE statement
Expansion	May require new registers or files	Additional tables and relationships

This comparison is conceptual. Actual performance depends on the size of the institution, the quality of the existing process, and how the database is implemented.

6.4 Expected impact

For a small academic dataset, the main benefit is not necessarily speed but consistency and ease of demonstration. The project illustrates how information can be structured and queried using a standard relational model.

In a larger institution, an appropriately designed and secured system could support more reliable record maintenance, provided that authentication, authorization, backups, auditing, and appropriate application interfaces are implemented.

7. FEASIBILITY STUDY

A feasibility study evaluates whether a proposed system is practical to develop and use.

7.1 Technical feasibility

The project is technically feasible because it uses widely available development tools. MySQL supports the required SQL features, and Visual Studio Code can be used to edit SQL files and connect to a MySQL server through a suitable extension.

The database requires only a small number of tables and sample records. No specialized hardware or advanced programming framework is necessary.

The project can be implemented on a compatible personal computer with MySQL Server installed and running.

7.2 Economic feasibility

The project can be developed at low cost using available software tools. MySQL Community Edition, Visual Studio Code, and GitHub provide suitable options for an academic prototype without requiring the purchase of a commercial database

license.

The primary cost is the time spent designing the schema, writing SQL queries, testing results, and preparing documentation.

7.3 Operational feasibility

The system is operationally feasible because the commands are relatively simple. Users with basic SQL knowledge can execute queries to view and modify records.

However, a production system would need a user-friendly interface, user permissions, documented procedures, and training for staff members.

7.4 Schedule feasibility

The project can be divided into manageable stages:

Stage	Activity
1	Identify the problem and requirements
2	Design tables and relationships
3	Write and execute SQL statements
4	Test the queries and verify outputs
5	Capture screenshots and prepare the report
6	Upload files to GitHub and verify submission

7.5 Conclusion of feasibility study

The proposed system is feasible as a student-level DBMS project. It uses basic relational database concepts, requires minimal resources, and provides sufficient scope to demonstrate SQL operations and data integrity.

Its feasibility for real institutional deployment would require additional analysis of security, privacy, performance, reliability, and administrative requirements.

8. MODULES AND FUNCTIONAL REQUIREMENTS

8.1 Student registration module

This module is responsible for inserting student details into the Students table.

Each record includes a student ID, student name, email, age, course ID, and marks. The primary key prevents duplicate student IDs, while other constraints validate selected fields.

The module is demonstrated through INSERT statements.

8.2 Course management module

This module stores course IDs and course names in the Courses table.

Each course has a unique ID. The UNIQUE constraint on course_name prevents duplicate course names in the sample schema.

Course records must be created before inserting students who reference them.

8.3 Student record retrieval module

This module displays all student records using SELECT. It also supports searching for a specific student using a WHERE condition.

Such queries allow an administrator to locate relevant records without displaying unrelated information.

8.4 Update module

The update module modifies existing records. For example, the marks of a student can be changed by specifying the student's ID in an UPDATE statement.

The WHERE clause is essential because an UPDATE statement without an appropriate condition can modify multiple records.

8.5 Performance analysis module

This module uses filtering, sorting, and aggregate functions.

A WHERE condition identifies students scoring above a specified threshold. ORDER BY sorts students by marks. AVG calculates the mean of the marks stored in the table.

8.6 Reporting module

The reporting module combines information from the Students and Courses tables through JOIN operations.

It can also count the students associated with each course by using COUNT and GROUP BY. A LEFT JOIN ensures that courses with no associated students can still appear in the course-count report.

8.7 Deletion module

This module removes a selected student record using DELETE and a WHERE condition.

Deletion should be performed only when authorized. In a real system, it may be preferable to maintain an inactive status or archival record instead of permanently deleting information.

8.8 Functional requirements

The functional requirements describe what the system must do:

ID	Requirement
FR1	Create a database for student management.
FR2	Store course details.
FR3	Store student records.
FR4	Enforce unique student and course identifiers.
FR5	Validate selected fields through constraints.
FR6	Display all student records.
FR7	Search for a student by ID.
FR8	Update student marks.
FR9	Filter and sort students.
FR10	Calculate average marks.
FR11	Retrieve student and course information together.
FR12	Count students in each course.
FR13	Delete a selected student record.

9. NON-FUNCTIONAL REQUIREMENTS

Non-functional requirements describe the qualities and operating conditions expected from the system.

9.1 Reliability

The database should store records accurately and return consistent results for correctly executed queries. Primary keys, foreign keys, and constraints help maintain reliable data.

9.2 Usability

The system should be understandable to users with basic SQL knowledge. Clear table names and meaningful column names make queries easier to read.

The current prototype uses a SQL editor rather than a graphical user interface, so usability depends partly on the user's familiarity with SQL.

9.3 Maintainability

The database structure should be documented so that another developer can understand the tables and their relationships. SQL scripts should be organized into logical sections and stored in a version-controlled repository.

9.4 Security

Real student data should only be accessible to authorized individuals. Database permissions should follow the principle of least privilege.

The sample project does not implement login functionality or separate application roles. Security would need to be added before real student records could be used.

9.5 Data integrity

The database should prevent duplicate primary keys, invalid course references, missing required names, duplicate non-NULL email values, and ages that violate the defined CHECK constraint.

These rules protect the structure and consistency of the stored records.

9.6 Portability

The SQL script uses MySQL-compatible syntax. It can be executed on a compatible MySQL installation, although the exact steps for connecting the editor may differ across operating systems and extensions.

9.7 Performance

For the small sample dataset, the queries should be straightforward to execute. Large-scale deployment would require performance evaluation, appropriate indexes, query optimization, and realistic testing with larger datasets.

9.8 Privacy

Student information can contain personal data. A real implementation should collect only necessary information, restrict access, protect backups, and follow the institution's applicable privacy and data-handling policies.

The records in this project are fictional examples created for academic demonstration.

10. DBMS CONCEPTS USED IN THE PROJECT

10.1 Tables

Tables are the primary structures in a relational database. Each table stores information about a particular entity.

The Students table stores information about individual students, while the Courses table stores information about courses.

10.2 Primary key

A primary key uniquely identifies each row in a table. It cannot contain duplicate values or NULL values.

In this project:

- Students.student_id is the primary key of Students.
- Courses.course_id is the primary key of Courses.

These keys allow records to be referenced unambiguously.

10.3 Foreign key

A foreign key establishes a relationship between two tables by referencing a primary or other eligible unique key.

The course_id column in Students references Courses.course_id.

This relationship ensures that a student record cannot reference a course that does not exist, as long as the foreign key constraint is enforced.

10.4 NOT NULL constraint

The NOT NULL constraint prevents a column from storing NULL values.

In this project, student_name and course_name are declared NOT NULL. Therefore, every inserted student must have a name, and every course must have a name.

10.5 UNIQUE constraint

The UNIQUE constraint prevents duplicate values in a column or combination of columns.

The project applies UNIQUE to email in Students and course_name in Courses. In MySQL, a UNIQUE column can generally contain multiple NULL values, so a UNIQUE constraint alone does not require an email address to be provided.

10.6 CHECK constraint

The CHECK constraint validates a condition on inserted or updated values.

The Students table uses CHECK (age >= 16). This means the database rejects an age below 16 when the constraint is enforced.

10.7 DEFAULT constraint

A DEFAULT value is used when an INSERT statement omits the corresponding column.

The marks column has DEFAULT 0. If a student is inserted without a marks value, the database uses zero. If the value is explicitly supplied, the supplied value is stored instead.

10.8 Aggregate functions

Aggregate functions perform calculations over a set of rows.

- AVG() calculates an average.
- COUNT() counts rows or non-NULL values, depending on the expression.
- SUM() calculates a total.
- MIN() returns the minimum value.
- MAX() returns the maximum value.

This project demonstrates AVG and COUNT.

10.9 JOIN operations

A JOIN combines records from related tables.

The INNER JOIN in the project displays students with their corresponding course information. The LEFT JOIN used in the course-count report also retains courses with zero students.

10.10 GROUP BY and ORDER BY

GROUP BY groups records for aggregate calculations. In the project, it groups student records by course name.

ORDER BY sorts query results. The performance query uses descending order so that higher marks appear first.

11. ER MODEL AND RELATIONSHIPS

11.1 Introduction to the ER model

An Entity-Relationship (ER) model represents the important entities in a database and the relationships between them.

It is useful during database design because it provides a conceptual view of the information before tables and SQL statements are created.

11.2 Entities

The project contains two main entities.

Entity 1: Student

Attributes:

- student_id — unique student identifier.
- student_name — student's name.
- email — email address.
- age — student's age.
- course_id — reference to the student's course.
- marks — numerical score.

Entity 2: Course

Attributes:

- course_id — unique course identifier.
- course_name — name of the course.

11.3 Relationship between Student and Course

A course can be associated with multiple students, while each student is assigned to one course in this simplified model.

This is a one-to-many relationship from Courses to Students.

The relationship is implemented through course_id. The primary key in Courses is referenced by the foreign key in Students.

11.4 ER diagram

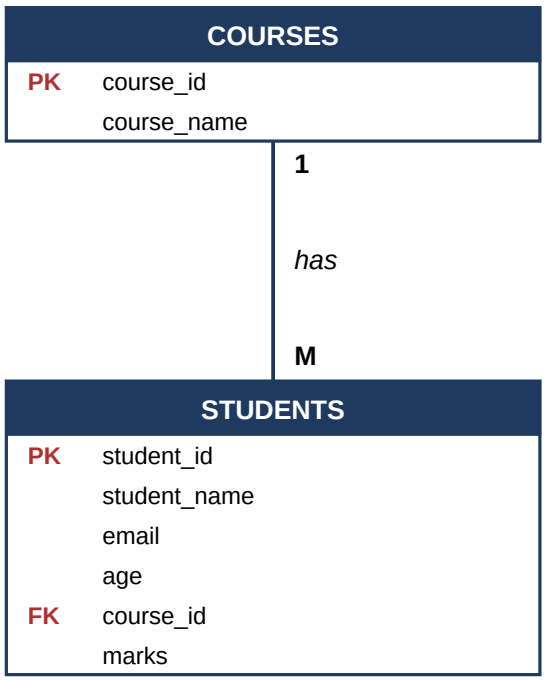


Figure 1: *Entity-Relationship Model of the Student Management System*

11.5 Advantages of the ER model

The ER model helps identify the entities and their attributes before implementation. It makes relationships easier to understand and provides a foundation for relational schema design.

It also helps identify unnecessary duplication and makes it easier to check whether each table represents a clearly defined entity.

11.6 Design limitation

The simplified ER model assumes that each student belongs to only one course. If a student can enrol in several courses, the design should be expanded by introducing a StudentCourse or Enrolment table.

This demonstrates that the database structure must reflect the actual rules of the institution being modelled.

12. RELATIONAL SCHEMA AND DATA DICTIONARY

12.1 Relational schema

The relational schema defines the structure of the database tables.

COURSES

Courses(course_id, course_name)

- Primary key: course_id
- Constraints: course_name is NOT NULL and UNIQUE.

STUDENTS

Students(student_id, student_name, email, age, course_id, marks)

- Primary key: student_id
- Foreign key: course_id references Courses(course_id).
- Constraints: student_name is NOT NULL, email is UNIQUE, age must be at least 16, and marks has a DEFAULT value of 0.

12.2 Data dictionary: Courses table

Attribute	Data type	Constraint	Description
course_id	INT	Primary key	Unique course number
course_name	VARCHAR(50)	NOT NULL, UNIQUE	Course name

12.3 Data dictionary: Students table

Attribute	Data type	Constraint	Description
student_id	INT	Primary key	Unique student number
student_name	VARCHAR(50)	NOT NULL	Student's name
email	VARCHAR(100)	UNIQUE	Student email address
age	INT	CHECK (age >= 16)	Student's age
course_id	INT	Foreign key	Associated course
marks	DECIMAL(5,2)	DEFAULT 0	Student's marks

12.4 Explanation of data types

- **INT:** Used to store integer values, such as IDs and age.
- **VARCHAR(n):** Used to store variable-length text up to the specified maximum length.
- **DECIMAL(5,2):** Used for precise numerical values with up to five total digits, including two digits after the decimal point. This allows values such as 95.00.

The selected data types reflect the nature of the information being stored.

12.5 Importance of the data dictionary

A data dictionary documents the meaning, type, and constraints of each column. It helps developers and database administrators understand the database without having to interpret every SQL statement.

It also supports maintenance and future development because changes can be evaluated against the documented data requirements.

13. NORMALIZATION AND DATA INTEGRITY

13.1 Meaning of normalization

Normalization is a database design process used to organize tables and reduce unnecessary data duplication and modification anomalies.

A well-normalized design separates logically different entities into appropriate tables and establishes relationships through keys.

13.2 First Normal Form (1NF)

First Normal Form requires each column to contain a single value for each row and discourages repeating groups within a table.

In the proposed design, each student record contains one student name, one email field, one age, one course reference, and one marks value. The course table contains one course ID and one course name per record.

The sample schema therefore follows the basic structure of 1NF.

13.3 Second Normal Form (2NF)

Second Normal Form requires a table to be in 1NF and every non-key attribute to depend on the whole candidate key rather than only part of a composite key.

In both project tables, the primary keys are single-column keys. Therefore, there is no partial dependency on part of a composite primary key in this schema.

The design satisfies the basic 2NF requirement under these assumptions.

13.4 Third Normal Form (3NF)

Third Normal Form aims to eliminate inappropriate transitive dependencies among non-key attributes.

For example, course_name depends on course_id. It is therefore stored in Courses rather than repeated as a separate course-name field in Students.

The Students table stores course_id as a reference. When the course name is needed, the database retrieves it through a JOIN.

This separation reduces duplication and makes course-name changes easier to manage.

13.5 Update anomaly

If a course name were stored repeatedly in every student record, renaming that course would require updating multiple rows. If some rows were not updated, the database could contain inconsistent course names.

Keeping course names in the Courses table avoids this particular problem.

13.6 Insertion anomaly

If course details were stored only in student records, adding a new course without enrolling a student could be difficult or impossible without inserting incomplete data.

The separate Courses table allows a course to be recorded independently of its student enrolments.

13.7 Deletion anomaly

If course information existed only in student rows, deleting the last student in a course might accidentally remove the only copy of the course information.

Separating the entities allows a course record to remain even if no students are currently associated with it.

13.8 Referential integrity

Referential integrity ensures that a foreign key references a valid record in the related table, subject to the configured foreign key rules.

In this project, a student cannot be inserted with a course_id that does not exist in Courses.

13.9 Domain integrity

Domain integrity refers to the validity of values stored in columns. Data types and constraints help enforce appropriate values.

The age CHECK constraint and the NOT NULL constraint on student_name are examples of domain-related validation.

13.10 Conclusion of normalization

The two-table design is appropriate for the simplified project scenario and demonstrates the basic ideas behind normalization and integrity constraints. More complex institutional requirements would need further normalization analysis and potentially additional entities.

14. TOOLS, TECHNOLOGIES AND METHODOLOGY

14.1 MySQL

MySQL is a relational database management system used to store and manage structured data. It supports SQL queries, relational tables, keys, constraints, aggregate functions, and joins.

In this project, MySQL stores the Students and Courses tables and executes the queries used to demonstrate the system's functionality.

14.2 Visual Studio Code

Visual Studio Code is a source-code editor that can be extended with tools for working with different programming languages and databases.

It is used in this project to write and maintain the SQL script. A compatible MySQL extension can provide a connection interface for executing queries.

14.3 MySQL extension

A MySQL extension can connect the editor to a running MySQL server. The connection typically requires the server address, port, username, password, and database configuration.

The exact interface depends on the extension installed. The user should verify that the connection is active before executing the script.

14.4 GitHub

GitHub is an online platform for hosting repositories and managing changes to project files.

The project repository can contain the SQL script, README, ER diagram, and output screenshots. This makes the project easier to share, review, and maintain.

14.5 Development methodology

The project follows a simple sequential development process.

Stage	Name	Description
1	Requirement analysis	Identify the information to be stored and the operations the database must support.
2	Database design	Identify entities, attributes, keys, and relationships.
3	Schema implementation	Create the database and tables using SQL.
4	Data population	Insert the course records followed by the student records.
5	Query implementation	Write queries for retrieval, modification, filtering, aggregation, and joins.
6	Testing	Execute the statements and verify their outputs.
7	Documentation	Capture output screenshots and prepare the project report.
8	Version control	Upload the completed project files to GitHub.

14.6 Hardware and software requirements

Hardware requirements

- A laptop or desktop computer.
- Sufficient memory and storage for MySQL and the editor.
- An internet connection for downloading tools and uploading the project.

Software requirements

- A compatible operating system.
- MySQL Server.
- Visual Studio Code.
- A compatible MySQL extension, if queries are being run through VS Code.
- A web browser and GitHub account.

15. MINI PROJECT — DATABASE AND TABLE CREATION

15.1 Aim of the practical

The aim is to implement the proposed database design using SQL and demonstrate the creation of tables with appropriate data types, keys, and constraints.

15.2 Step 1: Create the database

```
SQL

CREATE DATABASE StudentManagement;
USE StudentManagement;
```

Explanation: CREATE DATABASE creates the database named StudentManagement. USE selects it as the active database for subsequent statements.

Expected result: The database is created, and subsequent SQL statements are executed in that database.

```
OUTPUT (MySQL CLIENT)

mysql> CREATE DATABASE StudentManagement;
Query OK, 1 row affected (0.01 sec)

mysql> USE StudentManagement;
Database changed
```

Figure 1: *Database creation and selection*

15.3 Step 2: Create the Courses table

```
SQL

CREATE TABLE Courses (
    course_id INT PRIMARY KEY,
    course_name VARCHAR(50) NOT NULL UNIQUE
);

INSERT INTO Courses VALUES
(101, 'BCA Data Science'),
(102, 'BCA'),
(103, 'BBA');
```

Explanation: The Courses table stores the unique ID and name of each course. The primary key identifies each course, NOT NULL requires a course name, and UNIQUE prevents duplicate course names. The INSERT statement adds three sample course records.

Verification query:

```
SQL

SELECT * FROM Courses;
```

```
OUTPUT (MySQL CLIENT)

mysql> CREATE TABLE Courses (...);
Query OK, 0 rows affected (0.04 sec)

mysql> INSERT INTO Courses VALUES (101,'BCA Data Science'),(102,'BCA'),(103,'BBA');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM Courses;
+-----------+-----------------+
| course_id | course_name     |
+-----------+-----------------+
|       101 | BCA Data Science |
|       102 | BCA             |
|       103 | BBA             |
+-----------+-----------------+
3 rows in set (0.00 sec)
```

Figure 2: *Courses table creation and output*

15.4 Step 3: Create the Students table

```sql
CREATE TABLE Students (
    student_id INT PRIMARY KEY,
    student_name VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE,
    age INT CHECK (age >= 16),
    course_id INT,
    marks DECIMAL(5,2) DEFAULT 0,
    FOREIGN KEY (course_id)
        REFERENCES Courses(course_id)
);
```

Explanation: The Students table stores student details. The primary key identifies each student. The foreign key establishes a relationship with the Courses table. Other constraints validate the name, email, age, and default marks value.

Expected result: The Students table is created with the specified columns and constraints. The structure can be verified with DESCRIBE:

```sql
DESCRIBE Students;
```

```
mysql> CREATE TABLE Students (...);
Query OK, 0 rows affected (0.05 sec)

mysql> DESCRIBE Students;
+--------------+--------------+------+-----+---------+-------+
| Field        | Type         | Null | Key | Default | Extra |
+--------------+--------------+------+-----+---------+-------+
| student_id   | int          | NO   | PRI | NULL    |       |
| student_name | varchar(50)  | NO   |     | NULL    |       |
| email        | varchar(100) | YES  | UNI | NULL    |       |
| age          | int          | YES  |     | NULL    |       |
| course_id    | int          | YES  | MUL | NULL    |       |
| marks        | decimal(5,2) | YES  |     | 0.00    |       |
+--------------+--------------+------+-----+---------+-------+
6 rows in set (0.00 sec)
```

Figure 3: *Students table creation and structure*

15.5 Step 4: Insert student records

```sql
INSERT INTO Students VALUES
(1, 'Aarav Sharma', 'aarav@gmail.com', 19, 101, 85),
(2, 'Priya Verma', 'priya@gmail.com', 18, 102, 92),
(3, 'Rahul Singh', 'rahul@gmail.com', 20, 101, 76),
(4, 'Simran Kaur', 'simran@gmail.com', 19, 103, 88),
(5, 'Aditya Kumar', 'aditya@gmail.com', 18, 102, 81);
```

Explanation: This statement inserts five fictional student records. Each course_id refers to an existing record in Courses.

Expected result: Five rows are inserted successfully if the tables are empty and all constraints are satisfied.

```
mysql> INSERT INTO Students VALUES (1,'Aarav Sharma',...), ... (5,'Aditya Kumar',...);
Query OK, 5 rows affected (0.01 sec)
Records: 5  Duplicates: 0  Warnings: 0
```

Figure 4: *Student record insertion*

15.6 Practical observation

The first practical stage demonstrates that a relational database can be created and populated using SQL. The successful creation of both tables confirms that the basic schema has been implemented.

The foreign key also demonstrates how relational integrity is enforced during data insertion.

16. MINI PROJECT — RETRIEVAL AND UPDATING

16.1 Display all student records

```sql
SELECT * FROM Students;
```

Purpose: Displays all columns and records stored in the Students table.

Expected output before modification:

```
mysql> SELECT * FROM Students;
+------------+--------------+------------------+-----+-----------+-------+
| student_id | student_name | email            | age | course_id | marks |
+------------+--------------+------------------+-----+-----------+-------+
|          1 | Aarav Sharma | aarav@gmail.com  |  19 |       101 | 85.00 |
|          2 | Priya Verma  | priya@gmail.com  |  18 |       102 | 92.00 |
|          3 | Rahul Singh  | rahul@gmail.com  |  20 |       101 | 76.00 |
|          4 | Simran Kaur  | simran@gmail.com |  19 |       103 | 88.00 |
|          5 | Aditya Kumar | aditya@gmail.com |  18 |       102 | 81.00 |
+------------+--------------+------------------+-----+-----------+-------+
5 rows in set (0.00 sec)
```

Figure 5: *Display all students*

16.2 Search for a particular student

```sql
SELECT * FROM Students
WHERE student_id = 2;
```

Purpose: Retrieves the record for the student whose ID is 2.

Expected result: One record is displayed, belonging to Priya Verma.

```
mysql> SELECT * FROM Students WHERE student_id = 2;
+------------+--------------+-----------------+-----+-----------+-------+
| student_id | student_name | email           | age | course_id | marks |
+------------+--------------+-----------------+-----+-----------+-------+
|          2 | Priya Verma  | priya@gmail.com |  18 |       102 | 92.00 |
+------------+--------------+-----------------+-----+-----------+-------+
1 row in set (0.00 sec)
```

Figure 6: *Search student by ID*

16.3 Update student marks

```sql
UPDATE Students
SET marks = 95
WHERE student_id = 2;

SELECT student_id, student_name, marks
FROM Students
WHERE student_id = 2;
```

Purpose: Changes Priya Verma's marks from 92 to 95 and retrieves the updated record.

Expected output:

```
mysql> UPDATE Students SET marks = 95 WHERE student_id = 2;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT student_id, student_name, marks FROM Students WHERE student_id = 2;
+------------+--------------+-------+
| student_id | student_name | marks |
+------------+--------------+-------+
|          2 | Priya Verma  | 95.00 |
+------------+--------------+-------+
1 row in set (0.00 sec)
```

Figure 7: *Updated student marks*

16.4 Importance of the WHERE clause

The WHERE clause specifies which records should be affected by a query.

For SELECT, it limits which records are displayed. For UPDATE and DELETE, it is especially important because it prevents unintended changes to unrelated records.

For example, removing the WHERE clause from the UPDATE statement could update the marks of every student. Therefore, conditions should be checked carefully before executing modification statements.

16.5 Practical observation

The retrieval and update queries demonstrate how the database can locate and modify individual records without recreating the entire table.

The results also show why unique identifiers are useful for accurate record selection.

17. MINI PROJECT — FILTERING, AGGREGATION AND JOINS

17.1 Display students scoring above 80

SQL

```
SELECT student_name, marks
FROM Students
WHERE marks > 80
ORDER BY marks DESC;
```

Purpose: Selects students whose marks are greater than 80 and sorts the results from highest to lowest.

Because Priya's marks were updated to 95, the expected output is:

OUTPUT (MySQL CLIENT)

```
mysql> SELECT student_name, marks FROM Students WHERE marks > 80 ORDER BY marks DESC;
+--------------+-------+
| student_name | marks |
+--------------+-------+
| Priya Verma  | 95.00 |
| Simran Kaur  | 88.00 |
| Aarav Sharma | 85.00 |
| Aditya Kumar | 81.00 |
+--------------+-------+
4 rows in set (0.00 sec)
```

Figure 8: *Students scoring above 80*

17.2 Calculate average marks

SQL

```
SELECT AVG(marks) AS average_marks
FROM Students;
```

Purpose: Calculates the arithmetic mean of the marks stored in the table.

After Priya's marks are updated, the values are 85, 95, 76, 88, and 81. Average: (85 + 95 + 76 + 88 + 81) / 5 = 85

Expected output:

OUTPUT (MySQL CLIENT)

```
mysql> SELECT AVG(marks) AS average_marks FROM Students;
+---------------+
| average_marks |
+---------------+
|     85.000000 |
+---------------+
1 row in set (0.00 sec)
```

Figure 9: *Average marks*

The number of displayed decimal places may vary depending on the SQL client and display formatting.

17.3 Display student and course details using INNER JOIN

```sql
SELECT Students.student_name,
       Courses.course_name,
       Students.marks
FROM Students
INNER JOIN Courses
ON Students.course_id = Courses.course_id;
```

Purpose: Combines the student and course tables using their related course_id values.

Expected output:

```
mysql> SELECT Students.student_name, Courses.course_name, Students.marks
    -> FROM Students INNER JOIN Courses
    -> ON Students.course_id = Courses.course_id;
+--------------+------------------+-------+
| student_name | course_name      | marks |
+--------------+------------------+-------+
| Aarav Sharma | BCA Data Science | 85.00 |
| Priya Verma  | BCA              | 95.00 |
| Rahul Singh  | BCA Data Science | 76.00 |
| Simran Kaur  | BBA              | 88.00 |
| Aditya Kumar | BCA              | 81.00 |
+--------------+------------------+-------+
5 rows in set (0.00 sec)
```

Figure 10: *Student and course details*

17.4 Count students in each course

```sql
SELECT Courses.course_name,
       COUNT(Students.student_id) AS total_students
FROM Courses
LEFT JOIN Students
ON Courses.course_id = Students.course_id
GROUP BY Courses.course_name;
```

Purpose: Counts the students associated with each course.

Expected output:

```
mysql> SELECT Courses.course_name, COUNT(Students.student_id) AS total_students
    -> FROM Courses LEFT JOIN Students
    -> ON Courses.course_id = Students.course_id
    -> GROUP BY Courses.course_name;
+------------------+----------------+
| course_name      | total_students |
+------------------+----------------+
| BCA Data Science |              2 |
| BCA              |              2 |
| BBA              |              1 |
+------------------+----------------+
3 rows in set (0.00 sec)
```

Figure 11: *Course-wise student count*

Explanation: LEFT JOIN retains every course even if it has no matching student. COUNT(Students.student_id) counts the matching non-NULL student IDs. GROUP BY generates one result for each course.

17.5 Practical observation

The filtering query identifies selected records, AVG summarizes marks, INNER JOIN combines related information, and LEFT JOIN with GROUP BY produces a course-wise report.

These operations demonstrate how SQL can answer different information requirements using the same database.

18. MINI PROJECT — DELETE OPERATION AND FINAL RESULTS

18.1 Delete a student record

```sql
DELETE FROM Students
WHERE student_id = 5;
```

Purpose: Removes the student record with ID 5.

Expected result: One record is deleted, provided that the record exists and the operation is permitted.

```
mysql> DELETE FROM Students WHERE student_id = 5;
Query OK, 1 row affected (0.01 sec)
```

Figure 12: *Delete operation*

18.2 Display final student records

```sql
SELECT * FROM Students;
```

Expected output after deletion:

```
mysql> SELECT student_id, student_name, age, course_id, marks FROM Students;
+------------+--------------+-----+-----------+-------+
| student_id | student_name | age | course_id | marks |
+------------+--------------+-----+-----------+-------+
|          1 | Aarav Sharma |  19 |       101 | 85.00 |
|          2 | Priya Verma  |  18 |       102 | 95.00 |
|          3 | Rahul Singh  |  20 |       101 | 76.00 |
|          4 | Simran Kaur  |  19 |       103 | 88.00 |
+------------+--------------+-----+-----------+-------+
4 rows in set (0.00 sec)
```

Figure 13: *Final student table*

The final table contains four student records. The course records remain unchanged, as confirmed below:

```
mysql> SELECT * FROM Courses;
+-----------+-----------------+
| course_id | course_name     |
+-----------+-----------------+
|       101 | BCA Data Science |
|       102 | BCA             |
|       103 | BBA             |
+-----------+-----------------+
3 rows in set (0.00 sec)
```

18.3 Importance of deletion

The DELETE command removes records that meet a specified condition. It is useful for demonstrating data modification, but it must be used carefully.

In a real educational institution, student records may be subject to retention requirements. Permanent deletion may not be appropriate. A production system might use an inactive status or an archival process instead.

18.4 Final practical results

The completed SQL workflow demonstrates the following operations:

1. Database creation.
2. Table creation.
3. Insertion of course and student records.
4. Retrieval of all records.
5. Search by student ID.
6. Updating student marks.
7. Filtering and sorting based on marks.
8. Calculation of average marks.
9. Joining related tables.
10. Course-wise student counting.
11. Deletion of a selected record.

12. Verification of the final table.

The practical section provides evidence of how the theoretical database design can be implemented through SQL statements.

Note for submission: The outputs shown in this report are the expected results of the queries. Replace them (or add alongside them) with screenshots captured after executing the corresponding queries in your own MySQL environment.

19. TESTING AND OUTPUT VERIFICATION

19.1 Purpose of testing

Testing verifies whether the database behaves as expected when SQL statements are executed. It helps identify syntax errors, constraint violations, incorrect results, and unexpected changes to stored data.

For this project, testing is performed by executing SQL queries and comparing the actual results with the expected outputs.

19.2 Test cases

Test ID	Test performed	Expected result
TC01	Create the database	Database created successfully
TC02	Create Courses table	Table created
TC03	Create Students table	Table created with constraints
TC04	Insert three courses	Three course records inserted
TC05	Insert five students	Five student records inserted
TC06	Display all students	Five rows displayed
TC07	Search for student ID 2	Priya Verma's record displayed
TC08	Update student ID 2 marks	Marks changed to 95
TC09	Filter marks above 80	Four students displayed
TC10	Calculate average marks	Average equals 85
TC11	Join students and courses	Five matching records displayed
TC12	Count students by course	Counts of 2, 2, and 1 displayed
TC13	Delete student ID 5	One student record deleted
TC14	Display final records	Four student records displayed

19.3 Constraint testing

Constraints can be tested using controlled invalid inputs in a separate test session. Examples include:

- Attempting to insert a duplicate student_id.
- Attempting to insert a student with a missing required name.
- Attempting to insert an age below 16.
- Attempting to insert an unknown course_id.
- Attempting to insert a duplicate non-NULL email address.

When the relevant constraint is enforced and the statement violates it, MySQL should reject the invalid operation. These tests should be performed carefully so that the main project dataset remains unchanged.

Representative error messages from MySQL 8.x for each case are shown below (exact wording of constraint names may differ between versions):

Test A: Duplicate student_id

SQL

```
INSERT INTO Students VALUES (1, 'Test User', 'test1@gmail.com', 20, 101, 70);
```

OUTPUT (MySQL CLIENT)

```
ERROR 1062 (23000): Duplicate entry '1' for key 'students.PRIMARY'
```

Test B: Missing required name

SQL

```
INSERT INTO Students VALUES (6, NULL, 'test2@gmail.com', 20, 101, 70);
```

```
OUTPUT (MySQL CLIENT)

ERROR 1048 (23000): Column 'student_name' cannot be null
```

Test C: Age below 16

```
SQL

INSERT INTO Students VALUES (6, 'Test User', 'test3@gmail.com', 14, 101, 70);
```

```
OUTPUT (MySQL CLIENT)

ERROR 3819 (HY000): Check constraint 'students_chk_1' is violated.
```

Test D: Unknown course_id

```
SQL

INSERT INTO Students VALUES (6, 'Test User', 'test4@gmail.com', 20, 999, 70);
```

```
OUTPUT (MySQL CLIENT)

ERROR 1452 (23000): Cannot add or update a child row: a foreign key
constraint fails (`studentmanagement`.`students`, CONSTRAINT
`students_ibfk_1` FOREIGN KEY (`course_id`) REFERENCES `courses`
(`course_id`))
```

Test E: Duplicate email

```
SQL

INSERT INTO Students VALUES (6, 'Test User', 'aarav@gmail.com', 20, 101, 70);
```

```
OUTPUT (MySQL CLIENT)

ERROR 1062 (23000): Duplicate entry 'aarav@gmail.com' for key 'students.email'
```

Test F: DEFAULT value for marks

```
SQL

INSERT INTO Students (student_id, student_name, email, age, course_id)
VALUES (6, 'Default Test', 'default@gmail.com', 19, 103);

SELECT student_id, student_name, marks FROM Students WHERE student_id = 6;

DELETE FROM Students WHERE student_id = 6;
```

```
OUTPUT (MySQL CLIENT)

Query OK, 1 row affected (0.01 sec)

+------------+--------------+-------+
| student_id | student_name | marks |
+------------+--------------+-------+
|          6 | Default Test |  0.00 |
+------------+--------------+-------+
1 row in set (0.00 sec)

Query OK, 1 row affected (0.01 sec)
```

Test F confirms that the DEFAULT 0 value is applied when marks are omitted. The test row is deleted immediately so that the main dataset remains unchanged.

19.4 Output screenshot procedure

For every practical operation:

1. Open the SQL file in the editor.
2. Select and execute the required statement.
3. Confirm that the correct database is selected.
4. Check the query output or execution message.
5. Capture the editor and result grid in a readable screenshot.
6. Insert the screenshot under the matching heading.
7. Add a figure number and caption.
8. Verify that the screenshot corresponds to the expected result.

19.5 Testing limitations

The tests use a small sample dataset and do not measure performance under a large number of users. They also do not test concurrent transactions, authentication, backup recovery, or application-level security.

The project therefore verifies basic functional behavior rather than establishing production-level reliability.

20. RESULTS, LIMITATIONS AND FUTURE SCOPE

20.1 Results and findings

The project demonstrates that a basic Student Management System can be implemented using relational database concepts and SQL.

The database separates student information from course information and links the tables using a foreign key. This design supports retrieval of course details without storing the course name repeatedly in every student record.

The implementation also demonstrates the use of constraints to maintain valid records. SELECT, WHERE, ORDER BY, UPDATE, DELETE, AVG, COUNT, GROUP BY, INNER JOIN, and LEFT JOIN are used to satisfy the sample requirements.

The test cases define expected outcomes for the main operations and provide a structure for verifying the actual results captured from MySQL.

20.2 Limitations of the current system

- **Limited entities:** The database contains only student and course tables.
- **Single course per student:** The schema does not support multiple course registrations for a student.
- **Simplified marks field:** Only one marks value is stored for each student. There is no separate subject, examination, or semester table.
- **No graphical interface:** The system is operated through SQL statements rather than a dedicated application interface.
- **No authentication:** The prototype does not implement separate logins or roles for students, faculty, and administrators.
- **No attendance or fee module:** These functions are outside the current project scope.
- **Limited security controls:** The project does not include a complete access-control policy, audit logging, or backup strategy.
- **Small dataset:** The project uses only five sample students and three courses, so it does not establish performance at institutional scale.

20.3 Future enhancements

The system could be extended in several ways.

Attendance management

An Attendance table could record attendance against students, dates, and classes. Attendance summaries could then be generated using aggregate queries.

Examination management

Separate Subject, Examination, and Result tables could store marks for different subjects and semesters.

Multiple course enrolment

A StudentCourse table could be introduced to implement a many-to-many relationship between students and courses.

Fee management

A FeePayments table could maintain payment dates, amounts, and payment status.

Authentication and authorization

A secure application interface could provide different permissions for administrators, faculty members, and students.

Graphical user interface

A web or desktop interface could allow users to add and view records without writing SQL manually.

Backup and recovery

Automated backups and a documented recovery procedure could reduce the risk of permanent data loss.

Reporting and visualization

Reports could display course-wise enrolments, marks distributions, and academic summaries through charts and dashboards.

20.4 Broader learning outcomes

The project provides experience in translating an information management problem into a relational schema, defining keys and constraints, writing SQL queries, and verifying the resulting outputs.

It also illustrates that database design involves more than writing commands: it requires understanding the data, relationships, assumptions, validation rules, and limitations of the proposed solution.

21. CONCLUSION AND REFERENCES

21.1 Conclusion

The Student Management System using MySQL was designed as an academic project to demonstrate the application of database management principles to student record handling.

The project begins with a case study of the difficulties associated with maintaining student information manually. It identifies the need for structured data storage and proposes a relational database containing Students and Courses tables.

The theoretical section discusses the role of a DBMS, relational database concepts, project objectives, requirements, feasibility, the ER model, relational schema, normalization, and data integrity. The practical section demonstrates the implementation of these concepts through SQL statements.

The project uses primary keys and foreign keys to identify records and establish relationships. NOT NULL, UNIQUE, CHECK, and DEFAULT constraints provide basic data validation. SQL queries demonstrate record insertion, retrieval, updating, filtering, sorting, aggregation, joining, and deletion.

Testing is planned through defined test cases, expected outputs, and actual screenshots captured during query execution. The results demonstrate the usefulness of relational tables and SQL for structured information management.

Although the current system is limited to a small dataset and a basic database schema, it provides a foundation for future extensions such as attendance, examination, fee management, authentication, and reporting.

Overall, the project strengthens practical understanding of MySQL, SQL queries, relational database design, and the importance of maintaining accurate and consistent data. It also demonstrates how theoretical DBMS concepts can be applied to a realistic educational scenario.

21.2 References

1. Oracle. *MySQL 8.4 Reference Manual.* Available at: https://dev.mysql.com/doc/refman/8.4/en/
2. MySQL Documentation. *SQL Statements.* Available at: https://dev.mysql.com/doc/refman/8.4/en/sql-statements.html
3. MySQL Documentation. *CREATE TABLE Statement.* Available at: https://dev.mysql.com/doc/refman/8.4/en/create-table.html
4. MySQL Documentation. *Using Foreign Key Constraints.* Available at: https://dev.mysql.com/doc/refman/8.4/en/create-table-foreign-keys.html
5. Microsoft. *Visual Studio Code Documentation.* Available at: https://code.visualstudio.com/docs
6. GitHub Docs. *About Repositories.* Available at: https://docs.github.com/en/repositories
7. Elmasri, R. and Navathe, S. B. *Fundamentals of Database Systems.* Pearson.
8. Silberschatz, A., Korth, H. F., and Sudarshan, S. *Database System Concepts.* McGraw Hill.